КРАТКИЙ ИТОГ НА ОДНОЙ СТРАНИЦЕ

Постквантовая проверка криптографии: bdk-wallet @ 09c677305139

Клиент: Bitcoin Dev Kit (bdk_wallet) — публичный репозиторий (пассивная проверка) · Код: github.com/bitcoindevkit/bdk_wallet, коммит `09c677305139` · Проверено: 2026-10-29 18:40 UTC · Профиль риска: Криптобиржа, кастодиан или кошелёк

ПОСТКВАНТОВАЯ ЗРЕЛОСТЬ

Уровень 0 из 4

постквантовой криптографии нет

НАХОДКИ

0 Критическая **0** Высокая
2 Средняя **0** Низкая
0 Информационная

КВАНТОВЫЙ СРОК (МОСКА: X + Y − Z)

Подписи: 10 + 5 − 8 = 7 → **слишком поздно**

Следующий шаг: Проведите инвентаризацию: где классическая криптография с открытым ключом защищает долгоживущие данные и ключи, и выберите первую миграцию — обычно гибридный обмен ключами TLS и обёрнутые ключи.

ГЛАВНОЕ

Средняя QSI-2026-06654	**Квантово-уязвимый ECDSA: подпись транзакций и сообщений (6 файл(ов))** предварительно Что сделать: Следите за постквантовыми планами каждой сети, не используйте открытые ключи повторно и держите подписание готовым к постквантовым схемам...
Средняя QSI-2026-06653	**Квантово-уязвимый ECDSA: роль не определена (1 файл(ов))** предварительно Что сделать: Классифицируйте каждое использование, зафиксируйте сроки жизни данных и запланируйте миграцию на ML-DSA (FIPS 204) или SLH-DSA (FIPS 205)...

ЧТО ЗАЩИЩАЕТ КВАНТОВО-УЯЗВИМАЯ КРИПТОГРАФИЯ

Роль	Алгоритмы	Что сделать
Средняя подпись транзакций и сообщений	ECDSA	Следите за постквантовыми планами каждой сети, не используйте открытые ключи повторно и держите подписание...
Средняя роль не определена	ECDSA	Классифицируйте каждое использование, зафиксируйте сроки жизни данных и запланируйте миграцию на ML-DSA...

Статический анализ исходного кода и манифестов; проанализировано файлов: 28. Запросы к системам владельца не отправлялись.

Полный отчёт, SHA-256 содержимого: `b5aca581ea6518105c006631bb85a071e969e4986c59649d45daa4f0d385be01`

Подпись полного отчёта (report.sig.json): ключ Ed25519, отпечаток SHA-256 `5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`. bundle.sig.json содержит SHA-256 каждого файла комплекта и тоже подписан; verify-bundle проверяет его без сети.

Предварительные находки требуют подтверждения в коде; доказательства, границы каждой проверки и порядок повторной проверки — в полном отчёте.